# ECE786 Final Project Report

*Profile-Based Cache Bypassing*

Suhas Shastry Hebbe Sathish (200676039)

## ***Task 1: Cache Efficiency Analysis***

For Task 1, I evaluated the six required benchmarks: BFS, LPS, NQU, BP, HS, and LUD. I used the `ECE786_NCSU` configuration with the 1M instruction cap. Each benchmark was run twice: once with normal L1D behavior and once with full global-memory L1D bypass enabled.

The two settings were:

```
-gpgpu_gmem_skip_L1D 0   # baseline
-gpgpu_gmem_skip_L1D 1   # full L1D bypass for global memory
```

I extracted per-kernel IPC values from the simulator output. For benchmark-level classification, I used the final `gpu_tot_ipc`, since it represents cumulative IPC across the full benchmark execution rather than a simple average of kernel IPCs. I classified benchmarks as cache-unfriendly when full bypass improved IPC by roughly 3–4% or more.

### Task 1 Results

| ***Benchmark name*** | ***kernel_name*** | ***kernel_launch_uid*** | ***IPC without cache bypassing*** | ***IPC with cache bypassing*** | ***Percent change in IPC*** | ***Benchmark category*** |
|---|---|---|---|---|---|---|
| *BFS* | *Aggregated (all kernels)* | *1–9* | *24.2689* | *40.3648* | *+66.32%* | *Cache Unfriendly* |
| *LPS* | *_Z13GPU_laplace3diiiiPfS_* | *1* | *371.5598* | *545.6426* | *+46.85%* | *Cache Unfriendly* |
| *NQU* | *_Z24solve_nqueen_cuda_kerneliiPjS_S_S_i* | *1* | *34.2939* | *34.3879* | *+0.27%* | *Cache Insensitive* |
| *BP* | *Aggregated (2 kernels)* | *1–2* | *553.0369* | *581.3123* | *+5.11%* | *Cache Unfriendly* |
| *HS* | *_Z14calculate_tempiPfS_S_iiiiffffff* | *1* | *356.3531* | *366.1858* | *+2.76%* | *Cache Insensitive* |
| *LUD* | *Aggregated (all kernels)* | *1–46* | *41.8475* | *43.5609* | *+4.09%* | *Cache Unfriendly* |

*TABLE 1: CACHE EFFICIENCY ANALYSIS (AGGREGATED PER BENCHMARK)*

Based on this, I selected **BFS, LPS, BP, and LUD** for Task 2. NQU and HS were not used for Task 2 because their IPC changes were small.

The results also show that cache behavior is not uniform across all kernels. BFS is a good example: the early BFS kernels are cache-friendly, but the later kernels improve strongly with bypassing, so the overall benchmark is cache-unfriendly. BP is also slightly mixed, since its second kernel slows down with bypassing, but the benchmark-level IPC still improves because the dominant kernel benefits.

The complete per-kernel Task 1 table is included in:

`task1/parsed/task1_kernel_ipc.csv`

I also made sure the earlier PA3 hardcoded address-range bypass was disabled, so this task is a clean comparison controlled only by `-gpgpu_gmem_skip_L1D`.

# ***Task 2: Profile-Based Cache Bypassing***

For Task 2, I implemented a two-run profile-based bypass scheme. The first run profiles cache-block reuse, and the second run uses that profile to decide whether a global load should use or bypass L1D.

The bypass rule is as follows:

```
if reference_count < 3:
    bypass L1D
else:
    use L1D
```

## Profile Data Structure

I added a profile manager that tracks reference counts using the key:

`SM ID → kernel launch UID → cache-block address → reference count`

This keeps the profile per-SM and per-kernel, as required. Including the kernel launch UID is important for benchmarks like BFS and LUD because they launch many kernels with different memory behavior.

The profile uses cache-block addresses, not raw byte addresses. The block address is computed using the configured L1D line size, so all accesses to the same cache line map to the same profile entry.

The dumped profile follows the required group format:

```
< SM 0, kernel 1 : 0xADDR ref, 0xADDR ref, ... >
```

## Profile Generation Run

In the profiling run, L1D is still enabled:

```
-gpgpu_gmem_skip_L1D 0
-ece786_profile_mode 1
```

The profiling hook is placed in `ldst_unit::process_memory_access_queue_l1cache()` in `shader.cc`. Global load accesses that reach the normal L1D path were recorded. The counter is incremented only when the access is accepted/consumed, not on every stall retry. This avoids inflating reference counts when the same memory access waits in the queue for multiple cycles.

After simulation, the profile manager dumps the per-SM, per-kernel, per-block counters to the profile file.

## Profile-Based Bypass Run

In the second run, the simulator loads the profile before execution starts:

```
-ece786_use_profile_bypass 1
-ece786_bypass_threshold 3
```

For each global load, the bypass logic looks up the profiled reference count using the current `SM ID, kernel UID, and block address`. If the count is less than 3, the request bypasses L1D.

I used an aggressive missing-entry policy:

```
-ece786_missing_as_zero 1
```

This means missing profile entries are treated as count 0 and bypass L1D. I used this because the profile run and bypass run can have different timing, which may change block scheduling and SM assignment. For irregular workloads like BFS, treating unseen blocks as low reuse gave better behavior than forcing them into L1D.

The response path also needed fixing. I added a flag in *mem_fetch* to remember whether a request was profile bypassed. This flag is checked when the response returns, so a bypassed request does not incorrectly fill L1D on the way back.

## Modified Files

The modified simulator files are mirrored in:

*task2/modified_files/*

The files included are:

*gpu-sim.cc*
*gpu-sim.h*
*mem_fetch.cc*
*mem_fetch.h*
*shader.cc*

## Task 2 Results

Task 2 was run only on the cache-unfriendly benchmarks from Task 1: BFS, LPS, BP, and LUD.

| ***Benchmark*** | ***Baseline IPC*** | ***Full Bypass IPC*** | ***PBB IPC*** | ***PBB vs Baseline*** | ***PBB vs Full Bypass*** |
|---|---|---|---|---|---|
| *BFS* | *24.27* | *40.36* | *29.21* | *+20.36%* | *-27.63%* |
| *LPS* | *371.56* | *545.64* | *533.36* | *+43.55%* | *-2.25%* |
| *BP* | *553.04* | *581.31* | *591.51* | *+6.96%* | *+1.75%* |
| *LUD* | *41.85* | *43.56* | *43.22* | *+3.28%* | *-0.78%* |

*TABLE 2 : IPC COMPARISON OF TASK 2*

Profile-based bypass improves all four cache-unfriendly benchmarks over the baseline.

LPS is the cleanest result. PBB reaches 533.36 IPC, which is very close to the full-bypass result of 545.64 IPC. This suggests that LPS has a mostly low-reuse access stream, so bypassing most low-reference blocks is close to the ideal behavior.

BP is the strongest selective-bypass result. Full bypass improves BP, but profile-based bypass performs even better. This shows that blindly bypassing everything is not always best; BP still has some useful locality, and the profile-based policy preserves that while bypassing low-reuse traffic.

LUD improves moderately and comes close to full bypass. LUD has a mix of kernel behavior. The internal kernels benefit more from bypassing, while diagonal kernels are less sensitive.

BFS improves significantly over baseline, but it does not match full bypass. BFS has irregular memory behavior, and its kernel behavior changes across launches. The profiling run cannot perfectly predict the bypass run once memory timing changes. Honestly, I expected a bigger gain here. The profiling limitation hit BFS harder than the other benchmarks.

## How to Reproduce

The project can be run from the ***/accel-sim/accel-sim-framework*** path using:

```
./run_ece786_project.sh --full
```

Useful script modes are:

```
./run_ece786_project.sh --task1
./run_ece786_project.sh --task2
./run_ece786_project.sh --parse-only
./run_ece786_project.sh --summary
```

***--full*** builds the simulator, runs Task 1, runs Task 2, parses the logs, and prints the final summaries. ***--parse-only*** regenerates CSVs from existing logs.

The parsed results are written to:

```
task1/parsed/
task2/parsed/
```

Raw logs and generated profiles are stored in:

```
task1/raw_logs/
task2/raw_logs/
task2/profiles/
```

## Conclusion

Full L1D bypass works, but it is blunt. It removes cache pollution, but it also loses any useful L1D locality.

My Profile-based bypass implementation gives a more selective alternative. By using per-SM, per-kernel, per-block reference counts, it bypasses low-reuse blocks while keeping higher-reuse blocks cacheable. In my results, profile-based bypass improved all four cache-unfriendly benchmarks over the baseline. BP even outperformed full bypass, which shows the value of selective reuse-aware cache management.